シーケンシャルパターンに基づくセキュリティパッチの Root Cause Localization 手法

荒川 玲佳[1,a)]　大庭 翔介[1]　鐘本 楊[1]

概要：ソフトウェア脆弱性に関する既存研究では，セキュリティパッチの変更差分を脆弱性関連コードとして利用することが多い．しかし，セキュリティパッチには脆弱性修正とは無関係な変更が含まれるほか，不正動作を引き起こす脆弱性の発現箇所が変更差分に含まれない場合がある．特に，発現箇所に到達する値や状態を制御する上流の根本原因が修正される場合，変更箇所のみから発現箇所を特定することは困難である．そこで本研究では，セキュリティパッチの変更箇所を起点として発現箇所を特定する Root Cause Localization 手法を提案する．提案手法では，変更対象と防御メカニズムを組み合わせた新たな概念であるシーケンシャルパターンを導入し，修正箇所を分類する．次に，変更箇所を起点として Program Dependency Graph におけるデータ依存および制御依存関係を探索し，危険な API 呼出しや操作との照合により発現箇所候補を特定する．GitHub Advisory Database にある C/C++脆弱性 463 件と正解データ 153 件を用いて評価した結果，シーケンシャルパターンはグループ単位で約 90%の分類精度を示し，発現箇所の特定精度は約 25%であった．また，分析により変更箇所と発現箇所が異なる関数に存在することが 10%あることを明らかにした．既存手法にある変更差分のみを脆弱性関連コードとみなすアプローチでは発現箇所を見逃す可能性があり，本研究が問題視する，変更差分外に脆弱性の発現箇所が存在するケースを特定する必要性を示す．

キーワード：Root Cause localization, セキュリティパッチ，シーケンシャルパターン

Root Cause Localization in Security Patches based on Sequential Pattern Analysis

Reika Nishimura.Arakawa[1,a)]　Shosuke Oba[1]　Yo Kanemoto[1]

Abstract: Security patches are widely used to identify vulnerability-related code; however, vulnerability sinks may exist outside patch-modified code. We propose a vulnerability root cause localization approach that identifies such sinks from fix locations (FLs). Our approach classifies FLs into sequential patterns to infer sink locations and search directions, and then traverses data and control dependencies in a Program Dependency Graph (PDG) to identify sink candidates. We evaluate our approach using 463 C/C++ vulnerabilities and ground truth from 153 vulnerabilities. The sequential-pattern classification achieves approximately 90% accuracy at the group level, while sink localization achieves approximately 25% accuracy for 128 vulnerabilities where exploration was feasible. We also find that approximately 10% of the ground-truth vulnerabilities have FLs and sinks in different functions. These results highlight the importance of analyzing code beyond patch changes to identify vulnerability manifestation sites.

1. はじめに

ソフトウェアに含まれるセキュリティ脆弱性は，システムの安全性および信頼性を脅かす重要な問題である．脆弱性への迅速な対応を支援するため，これまで，ソースコー

[1] NTT 株式会社
NTT Social Informatics Laboratories
3-9-11 Midori-cho Musasino-shi Tokyo
[a)] reika.arakawa@ntt.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ドからの脆弱性検知，脆弱性の影響を受けるバージョンの特定，脆弱箇所への到達性判定，脆弱性に関連するコードのデータベース構築など，様々な研究が行われてきた．これらの研究では，脆弱性を修正したセキュリティパッチの変更差分からコードを抽出し，脆弱性に関連するコードの正解データとして利用する方法が広く用いられている．

研究課題． 既存手法には 2 つの課題がある．

第一に，セキュリティパッチには脆弱性修正と無関係な変更が含まれる．既存研究では，バグ修正コミットの 10% から 39% [9]，セキュリティパッチの約 36% [1] に無関係な変更が含まれることが報告されており，変更箇所（Fix Location; FL）のうち脆弱性修正に関与する部分の識別が必要である．

第二に，脆弱性の発現箇所（Sink Location; SL）は変更差分に含まれるとは限らず，上流の根本原因のみが修正される場合がある．そのため，FL のみの解析では，異なる関数や変更差分外の SL を見逃す可能性がある．したがって，脆弱性修正に関与する FL を識別し，FL から変更差分外の SL まで特定することが重要である．

本研究のアプローチ． 前述の通り，既存の脆弱性に関連する検知・解析・修正手法らは全て変更差分のみを脆弱性関連コードとみなすアプローチであり，発現箇所 SL を見逃す可能性がある．そのため，本研究では変更差分外に脆弱性の発現箇所が存在するケースを特定することを目的とする．

本研究では，セキュリティパッチの修正箇所（FL）から脆弱性の発現箇所（SL）を特定する Root Cause Localization 手法を提案する．提案手法は，変更対象と防御メカニズムを組み合わせた新たな概念であるシーケンシャルパターンを導入する．FL を分類し，パターンに応じて探索方向を決定し，Program Dependecy Graph 上のデータ依存・制御依存関係を追跡して危険な API や操作を発現箇所 SL 候補として特定する．これにより，変更差分の外に存在する SL を特定でき，脆弱性の原因と発現箇所の関係を明らかにすることで，脆弱性関連コードの抽出を精緻化する．

提案手法の有効性を検証するため，GitHub Advisory Database [5] の C/C++脆弱性 463 件とそこから作成した GroundTruth データ 153 件を用いて評価実験を行い，提案手法のシーケンシャルパターン分類および SL の特定の精度を評価し，提案手法が SL をどの程度正確に特定できるかを評価した．本研究の主な貢献を以下に示す．

- セキュリティパッチの変更差分を一律に脆弱性関連コードとして扱う既存手法に対して，脆弱性と無関係な変更の混入および悪影響の発現箇所が区別されないという課題に着目し，Root Cause Localization の問題を扱った．
- シーケンシャルパターンという新たな概念を導入し，パッチの変更差分を詳細に分類することで，ソースコードにおける FL, RC, SL のぞれぞれの位置関係を特定する手法を提案した．
- GitHub Advisory Database の C/C++の脆弱性 153 件を用いて提案手法を評価し，シーケンシャルパターンの分類精度はグループ許容で約 90%，SL 特定精度は探索が実行できた脆弱性 128 件中の約 25%の正解率であった．また，正解データの約 10%で FL と SL が異なる関数に存在することを明らかにした．

2. 背景

2.1 ソフトウェア脆弱性

ソフトウェア脆弱性とは，設計や実装上の欠陥に起因し，悪用によって情報漏洩や権限昇格，サービス妨害などを引き起こす可能性のある弱点である．公開された脆弱性には Common Vulnerabilities and Exposures（CVE） [3] によって識別子が付与され，National Vulnerability Database（NVD） [10] や GitHub Advisory Database [5] を通じて，概要や深刻度，影響を受ける製品などの情報が提供される．特に，今年 4 月から登場した Claude Mythos を始めとするフロンティア AI により発見される脆弱性の数が急増している [4]．同時に，適用すべきセキュリティパッチも同様に急増するため，システム運用者は今まで以上により瞬時にセキュリティパッチの分析や理解が求められる [2]．

2.2 セキュリティパッチを用いた脆弱性関連コードの特定

OSS で脆弱性が発見されると，開発者はセキュリティパッチを提供・公開する．修正前後のコード差分は脆弱性研究で広く利用されており，特に削除・変更されたコードは脆弱性ある箇所を特定する重要な手掛かりとなる．抽出されたコードは，機械学習による脆弱性検知の正解データ [6], [7], [13], [15]，脆弱性の導入時点や影響バージョンの推定 [8], [14]，到達性解析 [17]，脆弱性データセットの構築 [11] など様々な用途に利用されている．既存手法では，パッチ適用前に削除または変更された行を脆弱性あるコードとして扱うことが多く，脆弱性が発現するコードがパッチの変更差分に含まれることを前提としている．

2.3 課題

2.3.1 用語定義

脆弱性に関連するコードは，脆弱性の発生過程において異なる役割を持つ．本研究では，脆弱性に関連するコードを Fix Location (FL)，Root Cause（RC），Sink Location（SL）の三つの要素に区別し，以下のように定義する．

- FL はセキュリティパッチの変更差分から特定されるコード箇所
- RC は入力値の検証不備や不適切な境界値検査などの脆弱性を成立させる根本的な条件や安全性検証の欠落や誤りが存在するコード箇所
- SL は不正なメモリアクセスや危険な API の呼び出しなどの脆弱性が具体的な不正動作として最初に発現す

© 2026 Information Processing Society of Japan

るコード箇所

RC と SL は同一箇所に存在する場合もあれば，データフローや制御フローを介して異なる箇所に存在する場合もある．また，本研究では，脆弱性修正に関与する FL には，RC を修正する変更が含まれるものとする．

2.3.2 既存の脆弱性関連コード特定の課題

既存手法では，セキュリティパッチの変更差分から FL または，FL を含む関数を脆弱性関連コードとして特定する．しかし，SL に到達する値や状態を制御する上流の RC が修正される場合，SL は FL とは異なる関数や変更差分の外に存在するため，変更差分のみから特定できない．

具体例として，HTMLDOC で報告された脆弱性 CVE-2021-26259 の修正パッチでは，入力値やテーブル属性に対する検証処理が追加されている．一方，これらの値に起因する不正なメモリアクセスの SL は，FL を含む関数とは異なる `render_table_row()` に存在する．(付録 図 A·1) このような場合，FL を含む関数のみを解析しても，変更された値の伝播先に存在する SL を捉えることができない．

以上から，セキュリティパッチの変更差分に基づく脆弱性関連コードの特定には，以下の 2 つの課題がある．

課題 1：FL に脆弱性修正と無関係なコードが含まれる場合がある． セキュリティパッチに含まれるすべての変更が，脆弱性修正に直接関与するとは限らない．バグ修正コミットを対象とした調査では，変更の 10–39%がバグ修正とは無関係であることが報告されている [9]．また，OSS のセキュリティパッチを対象とした調査では，25 件中 9 件（約 36%）に脆弱性修正とは無関係な変更が含まれていた [1]．したがって，変更差分を一律に脆弱性関連コードとして扱うのではなく，脆弱性修正に関与する FL を識別する必要がある．

課題 2：FL に SL が含まれない場合がある． セキュリティパッチでは，SL そのものではなく，その上流に位置する RC が修正される場合がある．そのため，FL のみを解析対象とすると，変更差分の外に存在する SL を特定できない．脆弱性の発現箇所を特定するには，FL を起点としてデータ依存や制御依存などのプログラム依存関係を追跡し，SL まで探索する必要がある．

以上の課題に対して，本研究では，変更内容に基づいて脆弱性修正に関与する FL を識別し，さらに FL を起点としてプログラム依存関係を追跡することで，変更差分に含まれない SL を特定する手法を提案する．これにより，脆弱性修正と無関係な変更を除外するとともに，変更差分のみでは捉えられない SL を含めた脆弱性関連コードの特定を目指す．

3. 提案手法

3.1 提案手法の概要

本研究では，パッチの変更箇所（FL）を起点として，脆弱性の発現箇所（SL）を特定する手法を提案する．提案手法は，以下の二つの処理から構成される（図 1）．

処理 1 では FL に対する SL の相対位置と防御内容を特定する．FL の変更内容を正規化し，3 種類の変更操作と 8 種類の防御メカニズムの組合せから 9 種類のシーケンシャルパターンに分類する．このパターンに基づき，FL に対する SL の相対位置と探索方針を決定する．処理 2 では FL の近傍探索による SL を特定する．処理 1 で決定した探索方針に基づき，FL を起点として PDG 上のデータ依存および制御依存を探索する．危険 API または危険操作に該当するコードを SL 候補として特定し，各 FL から特定された候補を出力する．

3.2 処理 1：FL に対する SL の相対的な位置の特定

Step 1：FL の抽出と修正内容の要約 セキュリティパッチのコミット URL からパッチを取得し，diff を字句解析して hunk 単位の FL を抽出する．各 hunk は，hunk ヘッダー（`@@ -V,W +X,Y @@`）から次のヘッダーの直前までとする．抽出した FL から空行やコメントを除去した後，LLM を用いてデータ属性やコードの追加・削除・置換といった変更内容を自然言語で要約する．

Step 2：修正テンプレートによる正規化 各 FL を防御メカニズムに分類するため，差分から観測可能な 4 項目（`CHANGE_ACTION`，`ENTITY`，`SOURCE`，`TARGET`）からなる修正テンプレートに正規化する．各項目の詳細および具体例は付録 A.1 の表 A·1 に示す．

- `CHANGE_ACTION`：変更操作を表し，追加・削除・置換を `ADD`/`REMOVE`/`REPLACE` の 3 値で区別する．
- `ENTITY`：変更された構文要素を表し，条件や代入などの操作を `CALL`，`TYPE` 等の 11 値で分類する．
- `SOURCE`：FL で変更対象となる値の出所を表し，入力値や直前の関数呼び出しの戻り値などを `INDEX_SIZE` 等の 6 値で分類する．
- `TARGET`：FL で変更対象となるコード片を明示し，判定根拠および検証可能性を担保する．

Step 3：防御メカニズムの導出 正規化した項目を規則表 f に基づいて修正操作（`FIX_OPERATION`）に写像し，さらに規則 g で畳み込みを行って防御メカニズム（`F`）を一意に導出する．この写像を以下のように定義する（詳細は付録 A.1）．

$$\texttt{FIX_OPERATION} = f(\texttt{CHANGE_ACTION}, \texttt{ENTITY}, \texttt{SOURCE})$$

$$\texttt{F} = g(\texttt{FIX_OPERATION})$$

規則表 f は，実際のセキュリティパッチで観測された修正内容に基づいて定義する．複数の項目を組み合わせるのは，構文上類似した修正であっても，データの型や属性によって異なる防御メカニズムに分類するためである．

Step 4：シーケンシャルパターンの導出 防御メカニズム（`F`）と変更操作（`CHANGE_ACTION`）の組合せから，9 種類

© 2026 Information Processing Society of Japan

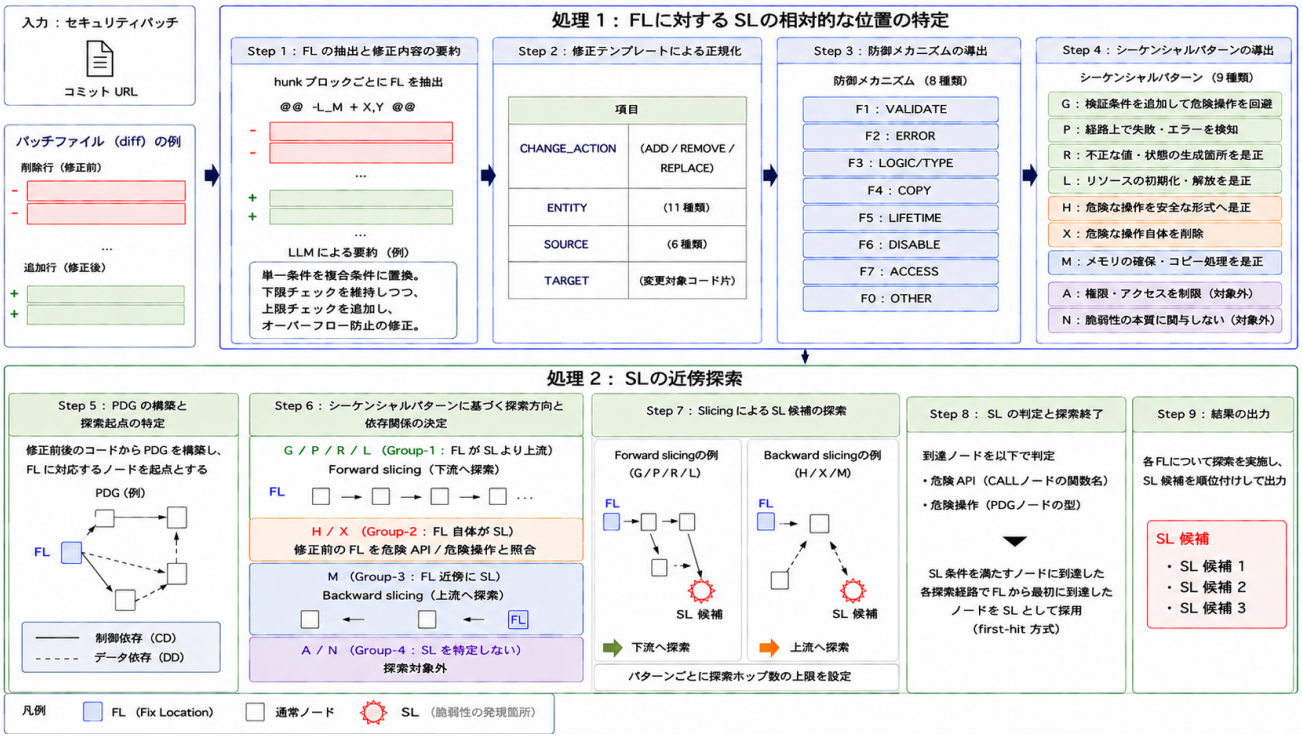


図 1　提案手法の全体像

のシーケンシャルパターンのうちどれに該当するかを特定する．シーケンシャルパターンを特定することで，後続の処理において FL に対する SL の相対位置や依存関係の推定が可能になる．分類を表 1 に示す．

Group-1：FL より下流に SL が存在するパターン． G（検証），P（エラー処理），R（値・状態の是正），L（リソース管理）が該当する．これらは SL に到達する前に原因となる値や状態を是正するため，FL は SL より上流に位置すると推定する．

Group-2：FL 自体に SL が存在するパターン． H（危険操作の是正），X（危険操作の削除）が該当する．これらは SL となる処理を直接修正または除去するため，FL 自体を SL と推定する．

Group-3：FL 近傍に SL が存在するパターン． M（メモリ操作の是正）が該当する．メモリサイズ等の定義と実際のメモリ操作が異なる箇所に存在するため，FL 近傍もしくは定義側の上流に SL が存在すると推定する．

Group-4：SL を特定しないパターン． A（アクセス制御）と N（脆弱性修正と無関係）が該当する．A は認可処理の欠落を原因とし，FL から SL を特定することが困難であるため，N は脆弱性修正に直接関与しないため，SL 探索の対象外とする．

3.3　処理 2：FL の近傍探索による SL 特定

処理 2 では，処理 1 で特定されたシーケンシャルパターンに基づいて SL の探索方向と依存関係を決定し，FL から PDG を探索する．探索先のコードを危険 API および危険操作と照合し，SL を特定する．なお，アクセス制御を表す A と脆弱性修正に関与しない N は探索対象外とする．各シーケンシャルパターンに応じた SL の探索および判定処理を表 2 に整理する．

Step 5：PDG の構築と探索起点の特定　修正前後のコードから，制御依存グラフ（CDG）とデータ依存グラフ（DDG）を統合した Program Dependency Graph（PDG）を構築する．次に，処理 1 の TARGET と PDG のノードを照合し，FL に対応するノードを探索起点として特定する．

Step 6：シーケンシャルパターンに基づく探索　G/P/R/L では FL が SL より上流に位置すると推定し，forward slicing を行う．G/P では制御依存（CD）とデータ依存（DD），R/L では DD を追跡する．一方，H/X/M では FL 自体または近傍に SL が存在すると推定し，修正前の FL を危険 API および危険操作と照合する．該当する場合は FL を SL と判定し，特に H では FL を優先的に SL 候補とする．

Step 7: Slicing による SL 候補の探索　G/P/R/L では forward slicing により FL の下流を探索する．H/X では FL から SL を直接特定できない場合，M では上流のデータ生成処理を特定するため，DD に基づく backward slicing を行う．また，過検出を抑制するため，パターンごとに探索ホップ数の上限を設定する．

Step 8: SL の判定と探索終了　探索した PDG ノードを，事前定義した危険 API および危険操作と照合する．判定対象は CWE に応じて絞り込み，対応するカテゴリがない場合はシーケンシャルパターンに応じたカテゴリを用いる．SL 条件を満たすノードに到達した場合，当該ノードを SL

© 2026 Information Processing Society of Japan

表 1　変更操作と防御メカニズムの組に応じて決定されるシーケンシャルパターン 9 つ

CHANGE_ACTION	F1 VALIDATE	F2 ERROR	F3 LOGIC/TYPE	F4 COPY	F5 LIFETIME	F6 DISABLE	F7 ACCESS	F0 OTHER
ADD	G_1	P_1	R_1	M_2	L_1	G_1	A_3	N_3
REMOVE	X_2	P_1	R_1	X_2	R_1	X_2	A_3	N_3
REPLACE	H_2	P_1	R_1	H_2	L_1	H_2	A_3	N_3

注釈：G (**G**uard)：検証条件を追加して脆弱性の顕在化を回避する；P (error **P**ropagate)：SL に到達する経路上で失敗やエラーを検知し，その結果を伝播する；R (**R**epair)：不正な値や状態の生成箇所を是正する；L (**L**ifecycle)：リソースの初期化や解放を是正する；H (**H**arden)：危険な操作自体を安全な形式へ是正する；X (e**X**clude)：危険な操作自体を削除する；M (**M**emory)：メモリの確保やコピー処理を是正する；A (**A**ccess)：権限やアクセスを制限する；N (**N**on-security)：脆弱性の本質に関与しない修正.
各シーケンシャルパターンに付与された下付き数字は，FL と SL の相対的な位置関係に基づくグループを表し，$_1$ は FL が SL より上流側に位置する Group-1，$_2$ は FL 自体が SL または SL の直近に位置する Group-2，$_3$ は FL から SL の位置を特定しない Group-3 を表す.

とし，各探索経路で FL から最初に到達した SL を採用する first-hit 方式により探索を終了する.

SL 条件を満たすノードに到達した場合，各探索経路で FL から最初に到達したノードを SL とする first-hit 方式により探索を終了し，過検出を抑制する.

4. 実験

手法の有効性評価のために，3 つのリサーチクエスチョン（RQ）を設定した.

RQ1 提案手法は脆弱性の FC のシーケンシャルパターンをどの程度正確に特定できるか

RQ2 提案手法は脆弱性の SL を正確に特定できるか

RQ3 脆弱性の種類によって提案手法の性能はどのように異なるか

4.1 データセットと Ground Truth の構築

データセット GitHub Advisory Database [5] から 2026 年 7 月 14 日時点で取得した C/C++の脆弱性を用いた．このうち，セキュリティパッチに対応するコミット URL を取得可能な CVE を抽出し，C 言語 398 件，C++言語 65 件の計 463 件を実験対象とした.

Ground Truth の構築 上記のデータセットから，脆弱性の説明文に SL の関数名およびファイル名が明示され，かつセキュリティパッチに脆弱性修正を含む CVE を抽出した．その結果，C 言語 133 件，C++言語 20 件の計 153 件を評価対象とした．各 FL に対して，目視により変更操作（E）と防御メカニズム（F）を付与し，その組合せからシーケンシャルパターンを分類して Ground Truth を構築した.

4.2 実験設定

実装および SL 判定に用いる危険 API・操作辞書の構築方法は，それぞれ付録 A.3.1，A.3.2 に示す.

4.2.1 評価メトリック

処理 1：シーケンシャルパターン Ground Truth と提案手法の同一 hunk に対するシーケンシャルパターンの一致率を算出する.

処理 2：SL 候補 提案手法が特定した SL 候補と Ground Truth の SL との一致率を算出する．ソースコードの行番号および SL コードの文字列が一致する場合を正解とする.

5. 評価

5.1 RQ1: 提案手法のシーケンシャルパターン特定精度

Ground Truth のシーケンシャルパターンに対し，提案手法の分類精度を (i) 9 種類のパターン単位と，(ii) 同一グループ内の誤分類を許容するグループ単位で評価した．結果を表 3 に示す．9 種類のシーケンシャルパターンの精度は C で 56.1%，C++で 65.0%であった．一方，グループ単位では Group-1 が C で 89.5%，C++で 90.0%と高い精度を示したのに対し，Group-2 はそれぞれ 66.7%，68.6%であった．この結果から，個々のパターンの識別には誤分類が生じるものの，SL の位置の分類単位では約 90%の精度で分類可能であることが分かった．なお，Group-3，4 は正解データに含まれないため評価対象外とした.

シーケンシャルパターン 9 分類の誤分類要因を分析した結果を表 4 に示す．最も多い誤分類は F1（VALIDATE）から F2（ERROR）で，17 FL（26.6%）を占めた．主な原因は，条件文の `SOURCE` が `RETURN_OF_CALL` または `ALLOC_RESULT` の場合，検査目的にかかわらず `CHECK_RETURN`（F2）に分類される規則にある．例えば，CVE-2023-23002 の NULL チェックも戻り値の検査として F2 に誤分類された．したがって，戻り値を検査する条件文では，値の出所だけでなく検査目的を考慮した分類が必要である.

5.2 RQ2: 提案手法の SL 特定の精度

データセットを用いて提案手法の処理 2 で特定された FL 234 件のうち，探索の到達状況の結果を表 5 に示す．SL 探索のためにスライスを実行できたのは 178 件 (76.1%)、シーケンシャルパターンが要求する種別のノードを特定できずに SL 探索できなかったのは 45 件 (19.2%)，パターン N（脆弱性二関係のない修正）のため探索対象外としたのは 11 件 (4.7%) であった。探索を実行した 178 件のうち，SL 候補を 1 件以上得たのは 110 件 (61.8%) であり，抽出された SL 候補は延べ 322 ノード（1 FL あたり平均 2.9,

© 2026 Information Processing Society of Japan

表 2　FL のシーケンシャルパターンに応じた SL の探索および判定処理

グループ	シーケンシャルパターン	探索方向	依存エッジ	SL の判定処理	
				危険 API 照合対象 (CALL ノード)	危険操作照合対象 (依存グラフのノード型)
Group1	G(検証条件を追加して危険操作を回避)	Forward	DD+CD	FL 自身/近傍ノード	探索先の近傍ノード
	P(SL 到達経路上で失敗・エラー検知)	Forward	DD+CD	FL 自身/近傍ノード	探索先の近傍ノード
	R(不正な値・状態の生成箇所を是正)	Forward	DD	FL 自身/近傍ノード	探索先の近傍ノード
	L(リソースの初期化・解放を是正)	Forward	DD	FL 自身/近傍ノード	探索先の近傍ノード
Group2	H(危険な操作自体を安全な形式へ是正)	FL 自身	–	FL 自身 *	FL 自身 *
	X(危険な操作自体を削除)	FL 自身	–	FL 自身 (削除行)*	FL 自身 (削除行)*
Group3	M(メモリの確保・コピー処理を是正)	Backward	DD	コピー：FL 近傍	Backward の近傍ノード
Group4	A(権限・アクセスを制限)	探索対象外	–	–	–
	N(脆弱性の本質に関与しない)	探索対象外	–	–	–

危険 API は CALL ノードの関数名を，危険操作は依存グラフのノード型（配列参照・ポインタ参照・確保・算術など）を照合する．いずれも対象 CVE の CWE に対応するカテゴリに限定し，該当カテゴリが未定義の場合はシーケンシャルパターンごとに定めた候補カテゴリを用いる．探索は最初に到達した SL で打ち切り（最近傍），パターンごとに定めたホップ数の上限で範囲を制限する．

表 3　処理 1 のシーケンシャルパターンの分類精度の結果

	分類	C	C++
(i)	シーケンシャルパターン（9 分類）	56.1%	65.0%
(ii)	Group-1 (G/P/R/L)	89.5%	90.0%
	Group-2 (H/X)	66.7%	68.6%

Group3,4 が正解データに存在せず，比較不能であったため省略する．

表 4　変更差分 FL の防御メカニズムの誤分類（上位 5）

正解 → 予測	件数	割合
F1 → F2（検査・制約 → 失敗検知・伝播）	17	26.6%
F5 → F1（生存期間 → 検査・制約）	5	7.8%
F4 → F3（転送・確保 → 計算・型）	5	7.8%
F1 → F3 (検査・制約 → 計算・型)	3	4.7%
F2 → F1 (失敗検知・伝播 → 検査・制約)	3	4.7%
合計	**64**	**100%**

中央値 2，最大 31）である．すなわち本手法は，1 つの修正 hunk に対して数個規模の少数候補に SL を絞り込んだ（付録 A.4.1）．

SL 候補の特定精度の評価は，提案手法の結果のうち正解データの hunk ヘッダと一致する 128 FL を対象とした．128 FL のうち，処理 2 で SL 候補を 1 件以上得たのは 68 FL であった．61 FL のうち，シーケンシャルパターンが厳密に一致していたのは 34 FL で，さらに SL が一致したのは 7 FL（34FL 中の 20.6%），Group 一致を許容する場合にシーケンシャルパターンが一致していたのは 44 FL で，さらに SL が一致したのは 11 FL（44 FL 中の 25.0%）であった．

5.3　RQ3: 脆弱性の種類に応じた SL 特定精度

SL 候補の特定精度を CWE の性質ごとに分析するため，正解データと対応する 128 FL を 5 つの CWE ファミリに分類した（表 6）．128 FL のうち 18 FL（14.1%）では探索を実行できず，68 FL で SL 候補が得られた．このうち 21 FL で正解 SL と完全一致し，一致率は対 GT で 16.4%，対候補で 30.9%であった．

表 5　SL 候補が正解データと一致した正解率

分類	評価条件	FL 数	一致 FL	正解率
(i)	パターン厳密一致	34	7	20.6%
(ii)	Group 一致を許容	44	11	25.0%

全体の約 44%を占めるバッファ境界系（CWE-119/787/125/120/121/122）は，対 GT 7.0%，対候補 12.1%と最も低かった．提案手法において，正解が関数呼び出しであるノードを探索すべきところを，ポインタ・添字参照など異なるノードを候補とする傾向が見られた．一方，NULL/ポインタ系（CWE-476/415/416）と資源/並行系（CWE-362/404/400/401）は，対候補でそれぞれ 61.5%，80.0%と高い精度を示した．数値系（CWE-190/191/369）は 33.3%（2/6）であり，入力検証系では正解 SL より粗粒度な式ノードが抽出され，完全一致に至らない傾向が見られた．

6.　考察

6.1　FL と SL が異なる関数に存在する脆弱性修正

今回の実験では，Joern の CDG が関数間の依存関係を解決できず，FL と SL が異なる関数に存在するケースを特定できなかった．そこで，FL と SL の関数を特定できた 153 CVE を対象に，両者の位置関係を分析した．結果を表 7 に示す．

153 CVE のうち 120 CVE（78.4%）では FL と SL が同一関数に存在した．一方，22 CVE では異なる関数に存在したが，7 CVE はクラス名の有無などの表記差であり，実

© 2026 Information Processing Society of Japan

表 6　CWE ファミリ別の SL 候補特定結果

CWE ファミリ	GT-FL	SL 候補あり	完全一致	一致率 (対 GT)	一致率 (対候補)
バッファ境界 (119/787/125/120/121/122)	57	33	4	7.0%	12.1%
NULL/ポインタ (476/415/416)	25	13	8	32.0%	61.5%
その他（CWE が空欄）	23	7	3	13.0%	42.9%
資源/並行 (362/404/400/401)	13	5	4	30.8%	80.0%
数値 (190/191/369)	6	6	2	33.3%	33.3%
入力検証 (20/1284/697)	4	4	0	0.0%	0.0%
合計	**128**	**68**	**21**	**16.4%**	**30.9%**

表 7　SL の関数と修正箇所 FL の関数の一致割合の結果

関係	行数	割合
一致	120	78.4%
不一致	15	9.8%
合計	**135**	**87.2%**

100%未満の理由は機械的に特定不能だった CVE が存在するため

質的な不一致は 15 CVE（9.8%）であった．これらは，主に二つに分類できた．第一に，FL が呼び出し元，SL が呼び出し先に存在するケース（6 CVE）であり，呼び出し先の戻り値を呼び出し元で検査する修正が見られた（例：CVE-2021-26259，CVE-2023-22999，CVE-2023-23002）．第二に，一つの脆弱性に対して複数箇所が変更され，SL を含む関数とは異なる関数にも FL が存在するケースである．

パターン別の不一致率は G が 14.3%，L と R が 16.7%，H が 17.2%，P が 26.9%であり，大きな差は見られなかった．このことから，関数をまたぐ修正は特定のシーケンシャルパターンに固有の現象ではないと考えられる．

以上から，FL は SL そのものではなく，SL へ到達するデータフロー経路上に存在し，その経路を遮断することで脆弱性を修正する場合がある．そのため，別の経路から同一 SL へ到達可能となれば，脆弱性が再び発現する可能性がある．また，FL を含む関数のみを対象とする既存手法では，異なる関数に存在する SL を見逃し，脆弱性関連コードを十分に特定できない可能性がある．

6.2　単一データフローから複数 sink に到達する脆弱性

正解データの分析を通じて，同一のデータフローから 2 つの SL が存在する脆弱性 CVE-2021-26259 と CVE-2021-23206 が存在した．これは HTMLDOC の ps-pdf.cxx において，同一の table 入力処理に起因する不正なデータが，異なる関数へ伝播し，それぞれ異なる SL で脆弱性を発現することを確認した．前者では `parse_table()` 内で Stack Buffer Overflow が発現する一方，後者では同処理で生成された table 情報が `render_table_row()` へ伝播し，Heap Buffer Overflow が発現する（図 A·1）．この結果は，1 つの入力に起因するデータフロー上に複数の SL が存在し得ることを示している．したがって，FL から最初に到達する SL のみを特定するだけでは，潜在的な脆弱性関連箇所を見落とす可能性があり，データフロー上の複数の SL を考慮した解析が必要であることを示している．

6.3　妥当性への脅威

適用可能な言語の制約　本研究は C/C++を対象としており，他言語への適用可能性は検証していない．言語によって構文やメモリ管理，脆弱性の発現形態が異なるため，シーケンシャルパターンや SL の判定基準をそのまま適用できるとは限らない．今後，他言語での評価を通じて一般性を検証する必要がある．

実験で使用された脆弱性のバイアス　Ground Truth の構築では，脆弱性の説明文から SL の関数名とファイル名を特定でき，かつセキュリティパッチを取得できる CVE を対象とした．そのため，評価対象には選択バイアスがあり，結果をすべての OSS 脆弱性に一般化できるとは限らない．

6.4　研究倫理

本研究で対象とする脆弱性は既にパッチが公開されている既知の脆弱性である．また，提案手法の実装および評価においては，外部公開されている脆弱性情報データベース，GitHub リポジトリ，OpenAI GPT に対して多数の API リクエストを送信した．しかし，いずれのアクセスにおいても，リミット上限を設定し，適切な範囲で実施した．

7.　関連研究

以下の領域では既知の脆弱性に関連するコードを特定するため，修正前後のコード差分から脆弱性関連コードを抽出する処理が共通的に利用される．

脆弱性検知　ReDeBug [6] は，セキュリティパッチによって削除されたコードを Bloom Filter で照合することで既知脆弱性を検知する．VUDDY [7] は，関数の文字列長と MD5 ハッシュ値を用いた完全一致のコードクローンを検出する．一方，脆弱コードの再利用時に修正や機能追加が行われると，完全一致に基づく手法では検知が困難となる．MVP [15] や MOVERY [13] は，コード変更や構文的差異を考慮することで，部分一致する脆弱コードを検知する．これらの手法はパッチの変更コードを脆弱性シグネチャとして利用する．

© 2026 Information Processing Society of Japan

脆弱性の影響を受けるバージョン特定　セキュリティパッチから脆弱性を特徴付けるコードを抽出し，影響を受けるバージョンを特定する手法も提案されている．VISION [14] は，脆弱性の説明文から重要なメソッドや文を選択し，重み付きの手続き間プログラム依存グラフとして表現することで重要な関数を絞り，該当関数を含むコミットタグを特定することで影響バージョンを特定する．ただし，脆弱性の説明文に依存するため，必要な情報が記載されていない脆弱性には適用が制限される．

PATEN [8] は，セキュリティパッチから AST レベルのシグネチャを構築し，脆弱性が未修正状態の API に絞って該当 API が存在するバージョンを特定する．このように，パッチから抽出したコードに基づいてバージョン特定が行われる．

脆弱箇所への到達性判定　脆弱コードが対象ソフトウェアに含まれていても，実行経路上から到達できなければ脆弱性が発現しない場合がある．このため，脆弱箇所への到達可能性を解析する手法が提案されている．VAScanner [17] は，パッチから抽出した脆弱な関数を起点に backward slicing を行い，呼び出しの始点となる API を特定し，対象ソフトウェアが該当 API を利用している場合に到達すると判定する．手法はパッチから抽出した FL を含む関数が抽出されているため，sink の関数を捉えきれていない．

脆弱性データベースの構築とパッチの精緻化　セキュリティパッチから脆弱性関連コードを抽出して CVE 等の情報と対応付けることで，コードレベルの脆弱性データベースを構築する研究も行われている．DISPATCH [11] は，バージョン間の 1 差分に存在する複数のコミット集合を入力に受け取り，複数の変更が混在するパッチを意味的な個別パッチへ分解し，セキュリティ変更と非セキュリティ変更を識別する．構文差分や細粒度のトークン情報，変更パターンを利用することで，リファクタリングなどが混在するパッチからセキュリティ関連の変更を抽出する．

パッチに混在する脆弱性修正とは無関係な変更を除外する研究も行われている．Arakawa ら [1] は，LLM の few-shot prompting とコード分割を組み合わせ，脆弱性修正とそれ以外の変更を分類する．25 件のパッチを用いた評価では accuracy 0.88 を達成し，9 件に脆弱性修正とは無関係な変更が含まれることを報告している．

8. おわりに

本研究では，セキュリティパッチの変更箇所から脆弱性の発現箇所を特定する手法を提案した．評価の結果，シーケンシャルパターンはグループ単位で約 90%の分類精度を示し，約 10%の CVE で FL と sink が異なる関数に存在することを明らかにした．今後は，他言語への拡張と発現箇所の特定精度の向上を図る．

参考文献

[1] Reika Nishimura Arakawa, Yo Kanemoto, and Mitsuaki Akiyama. Towards the identification of vulnerability-fixing code lines in oss security patches using lexical code segmentation and llms. In *39th Annual Conference on Data and Applications Security and Privacy, DBSec 2025*, pages 73–95, 2025.

[2] Cloud Security Alliance. Ai vulnerability storm. `https://labs.cloudsecurityalliance.org/mythos-ciso/`, visited on 2026-8-20.

[3] The MITRE Corporation. CVE™ Program Mission, 2026.

[4] FIRST Forecasting team. The 2026 vulnerability forecast update: Navigating the ai epoch. `https://www.first.org/blog/20260615-vulnerability-forecast-update`, visited on 2026-8-20.

[5] GitHub, Inc. GitHub Advisory Database, 2026.

[6] Jiyong Jang, Abeer Agrawal, and David Brumley. Redebug: Finding unpatched code clones in entire os distributions. In *2012 IEEE Symposium on Security and Privacy*, pages 48–62, 2012.

[7] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. Vuddy: A scalable approach for vulnerable code clone discovery. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 595–614, 2017.

[8] Li Lin, Jialin Ye, Chao Wang, and Rongxin Wu. Paten: Identifying unpatched third-party apis via fine-grained patch-enhanced ast-level signature. *IEEE Trans. Softw. Eng.*, 51(4):990–1006, April 2025.

[9] Hoan Anh Nguyen, Anh Tuan Nguyen, and Tien N. Nguyen. Filtering noise in mixed-purpose fixing commits to improve defect prediction and localization. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pages 138–147, 2013.

[10] NIST. National Vulnerability Database, 2026.

[11] Shiyu Sun, Yunlong Xing, Xinda Wang, Shu Wang, Qi Li, and Kun Sun. DISPATCH: Unraveling security patches from entangled code changes. In *34th USENIX Security Symposium*, 2025.

[12] David A. Wheeler. Flawfinder, 2026.

[13] Seunghoon Woo, Hyunji Hong, Eunjin Choi, and Heejo Lee. MOVERY: A precise approach for modified vulnerable code clone discovery from modified Open-Source software components. In *31st USENIX Security Symposium*, 2022.

[14] Susheng Wu, Ruisi Wang, Kaifeng Huang, Yiheng Cao, Wenyan Song, Zhuotong Zhou, Yiheng Huang, Bihuan Chen, and Xin Peng. Vision: Identifying affected library versions for open source software vulnerabilities. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1447–1459, 2024.

[15] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, and Wenchang Shi. MVP: Detecting vulnerabilities using Patch-Enhanced vulnerability signatures. In *29th USENIX Security Symposium*, pages 1165–1182, 2020.

[16] Fabian Yamaguchi. joernio/joern, 2026.

[17] Fangyuan Zhang, Lingling Fan, Sen Chen, Miaoying Cai, Sihan Xu, and Lida Zhao. Does the Vulnerability Threaten Our Projects? Automated Vulnerable API Detection for Third-Party Libraries. *IEEE Transactions on Software Engineering*, 2024.

© 2026 Information Processing Society of Japan

付　　録

A.1　提案手法の詳細

A.1.1　処理 1:FIX_OPERATION の操作定義語の選定

提案手法の処理 1 では，要約文を半構造化テンプレートへ変換することで，セキュリティパッチの変更内容を正規化する．テンプレートを構成する要素のうち，[FIX_OPERATION] は後段のシーケンシャルパターンの判定に用いられる重要な要素である．本研究では，事前に定義された分類体系を用いるのではなく，脆弱性の説明文に現れる修正動詞の共起関係から，[FIX_OPERATION] の分類体系をボトムアップに構築する．

具体的には，2026 年 7 月 14 日時点で Advisory Database から収集した 24,630 件の脆弱性説明文を対象とし，著者らが事前に定義したセキュリティ修正を表す 93 個の動詞（例：*escape*，*free*，*bound*）との文字列マッチングを行う．その結果，少なくとも 1 つの修正動詞を含む 14,399 件の CVE を抽出した．各 CVE を説明文に出現する修正動詞の集合として表現し，「説明文 × 動詞」の TF–IDF ベクトルを構築することで，修正動詞の共起関係を数値化する．得られたベクトルに K-means 法を適用し，17 個のクラスタに分類する．

各クラスタを，類似した修正動詞が共起するセキュリティ修正の意味的操作（FIX_OPERATION）として解釈し，クラスタ中心への寄与が大きい動詞（例：*bound*，*allocate*，*check*）に基づいて各操作を命名した．一方，修正動詞を含まない説明文は操作を特定できないため，OTHER として扱う．また，各クラスタの一貫性は，クラスタ内の動詞分布に対する正規化エントロピーを用いて評価する．

クラスタ数については，silhouette 係数がクラスタ数の増加に伴って単調に増加し，明確な最適値を示さなかった．そこで，本研究では各クラスタの意味的な解釈可能性を考慮してクラスタ数を 17 に設定した．このときの silhouette 係数は 0.2888 であった．本研究では，パッチが施す修正操作を 17 種の修正操作（FIX_OPERATION）として定義した（表 A·1）．

表 A·1　過去の脆弱性 24,630 件から構築した FIX_OPERATION

FIX_OPERATION	Definition
VALIDATE	未信頼な入力・引数の形式や値域を検査し，不正な入力を拒否
SANITIZE	危険な文字・要素を除去，無害化，エスケープ，またはエンコード
CLAMP	値，長さ，回数，深さを許容範囲に制限
CHECK_RETURN	関数の戻り値や成否コードを検査
PROPAGATE_ERROR	エラーを伝播し，安全側で処理を中断
GUARD_ACCESS	アクセス前に NULL，存在，状態を検査
CHANGE_CALCULATION	算術式や計算ロジックを修正
CHANGE_TYPE	型，ビット幅，符号，または型変換を修正
CHANGE_ALLOCATION	メモリやバッファの確保サイズ・方式を修正
CHANGE_COPY	データコピーや転送処理を修正
CLEANUP_RESOURCE	資源の解放や後処理を修正
INITIALIZE	変数やバッファを初期化
REPLACE_API	危険な API を安全な API やラッパーへ置換
RESTRICT_PERMISSION	権限，属性，可視性，スコープを制限
DISABLE_OPERATION	危険な機能や実行経路を無効化・削除
UPDATE_METADATA	設定，依存バージョン，メタ情報を更新
OTHER	上記のいずれにも該当しない操作

A.1.2　処理 1:FIX_OPERATION の決定ルール

提案手法の処理 1 の修正テンプレートから，修正操作（FIX_OPERATION）を決定する規則を表 A·2 に示す．

A.1.3　処理 1:17 種の FIX_OPERATION から防御メカニズム（F）への畳み込み規則 g

A.1.2 で決定された 17 種の FIX_OPERATION には，類似する操作内容が含まれるため，最終的に 8 つの防御メカニズムに著者らの分析で畳み込みを行うルールを以下の通りに定めた．

© 2026 Information Processing Society of Japan

表 A·2 修正テンプレート (CHANGE_ACTION, ENTITY, SOURCE) から FIX_OPERATION を決定する規則 f

ENTITY	CHANGE_ACTION	SOURCE	FIX_OPERATION	補足
CONDITION	ADD/REPLACE	PARAM_INPUT / FIELD_STATE	VALIDATE	入力の形式・値域を検査して拒否
	ADD/REPLACE	INDEX_SIZE	VALIDATE	境界・オーバーフローの前提検査
	ADD/REPLACE	PARAM_INPUT / FIELD_STATE (ポインタ)	GUARD_ACCESS	使用前の NULL/存在ガード
	ADD/REPLACE	RETURN_OF_CALL /ALLOC_RESULT	CHECK_RETURN	直前呼出/確保の成否を検査
	REMOVE	any	DISABLE_OPERATION	分岐・機能の削除
ASSIGNMENT	REPLACE	INDEX_SIZE / LITERAL (算術式)	CHANGE_CALCULATION	値を生む式そのものを是正
	ADD/REPLACE	INDEX_SIZE (min/-max/mask で上限)	CLAMP	値を範囲内へ丸める
	ADD	LITERAL (既定値/0)	INITIALIZE	変数の初期化
	REPLACE	PARAM_INPUT (無害化して代入)	SANITIZE	`x = escape(x)` 等
TYPE	REPLACE/ADD	any	CHANGE_TYPE	型・幅・符号の是正
CALL	REPLACE	RETURN_OF_CALL /LITERAL	REPLACE_API	危険 API を安全 API へ置換
	ADD	PARAM_INPUT	SANITIZE	sanitizer/escape 呼出の追加
	ADD	ALLOC_RESULT / FIELD_STATE (資源)	CLEANUP_RESOURCE	free/release の追加
	ADD	INDEX_SIZE (コピー長付与)	CHANGE_COPY	memcpy → memcpy_s/長さ制御
	REMOVE	any	DISABLE_OPERATION	危険呼出の削除
RETURN	ADD	RETURN_OF_CALL /FIELD_STATE	PROPAGATE_ERROR	早期リターンで失敗を伝播
MEMORY_OPERATION	REPLACE	INDEX_SIZE / ALLOC_RESULT	CHANGE_ALLOCATION	確保サイズ/方式の是正
	REPLACE	INDEX_SIZE (転送長)	CHANGE_COPY	バッファ転送の是正
	ADD	LITERAL (memset/0 クリア)	INITIALIZE	初期化
	ADD/REMOVE	ALLOC_RESULT (解放)	CLEANUP_RESOURCE	解放・後始末
CONTROL_FLOW	ADD	RETURN_OF_CALL	PROPAGATE_ERROR	goto cleanup 等
	REMOVE	any	DISABLE_OPERATION	経路・機能の無効化
	REPLACE	any	OTHER	並べ替え等
FUNCTION	REPLACE	any	REPLACE_API	実装の差し替え
	REMOVE	any	DISABLE_OPERATION	機能削除
PERMISSION	ADD/REPLACE	any	RESTRICT_PERMISSION	権限・属性・スコープの制限
DECLARATION	ADD	LITERAL	INITIALIZE	初期化付き宣言
	REPLACE	any (型変更)	CHANGE_TYPE	型宣言の是正
	any	メタ/依存/バージョン	UPDATE_METADATA	非機構的
OTHER	any	any	OTHER	分類不能

© 2026 Information Processing Society of Japan

表 A·3　防御メカニズムへの畳み込み規則 g

パターン	名称	修正操作
F1	VALIDATE_CONSTRAIN	VALIDATE, CLAMP, SANITIZE, GUARD_ACCESS
F2	ERROR_DETECT_PROPAGATE	CHECK_RETURN, PROPAGATE_ERROR
F3	FIX_COMPUTATION_TYPE	CHANGE_CALCULATION, CHANGE_TYPE
F4	FIX_TRANSFER_ALLOC	CHANGE_COPY, CHANGE_ALLOCATION
F5	LIFECYCLE	CLEANUP_RESOURCE, INITIALIZE
F6	DISABLE_REPLACE	DISABLE_OPERATION, REPLACE_API
F7	RESTRICT_PERMISSION	RESTRICT_PERMISSION
F0	NONESSENTIAL_UNKNOWN	UPDATE_METADATA, OTHER

A.2　提案手法の適用例

提案手法の適用例として，Listing 1 に CVE-2024-22862（CWE-190，CVSS=9.8）のパッチを示す．FFmpeg の JPEG XL パーサでは，ctx->skip の整数オーバーフローが発生する．FL は 6〜8 行目に該当し，JPEG XL コンテナから取得した Box サイズに対する上限チェックが追加された．この修正により ctx->skip の検証条件が置換され，11 行目の SL における加算が安全に行われるようになった．この例では，SL コードである `ctx->skip += size;` は，FL より下流に位置する．

提案手法を Listing 1 の CVE-2024-22862（CWE-190, CVSS=9.8）に適用した例を示す．

図 1　CVE-2024-22862 のパッチにおける RC と FL

```
1  static int skip_boxes(JXLParseContext *ctx,
2                        const uint8_t *buf, int buf_size){
3      ...
4      if (!size)
5          return AVERROR_INVALIDDATA;
6      /* invalid ISOBMFF size */
7-     if (size <= head_size + 4)
8+     if (size <= head_size + 4 ||
9+         size > INT_MAX - ctx->skip)
10             return AVERROR_INVALIDDATA;
11     ctx->skip += size;
12     ...
13 }
```

処理 1 の結果

要 約 文:　Replaces the single condition `if(size<=head_size+4)` with a compound condition `if(size<=head_size+4||size>INT_MAX-ctx->skip)`. The change keeps the original lower-bound check on `size` against `head_size+4` and adds an upper-bound check on `size` against `INT_MAX-ctx->skip`.

修正テンプレート:

表 A·4　処理 2 による SL 探索の到達状況

SL 探索可否の結果	FL 数	割合
SL の探索を実行可能	178	76.1%
パターンが要求種別ノードが特定不可	45	19.2%
パターン N のため探索対象外	11	4.7%
合計	234	100.0%

スライスを実行した 178 FL のうち，SL 候補を 1 件以上抽出できたのは 110 FL（61.8%）であった．抽出された SL 候補は延べ 322 ノードであり，1 FL あたりの候補数は平均 2.9，中央値 2，最大 31 であった．

```
[CHANGE_ACTION]=REPLACE [ENTITY]=ADD/CONDITION [
SOURCE]=INDEX_SIZE [TARGET]=size<=head_size+4||
size>INT_MAX-ctx->skip
```

シーケンシャルパターン:　上記の修正テンプレートに規則表 f を適用した場合，修正操作（FIX_OPERATION）は VALIDATE として導出され，畳み込み規則 g により防御メカニズム（F）は F2 となる．シーケンシャルパターンは表 1 に従って，Group-1 の G.

A.3　実験設定の詳細

A.3.1　実装

提案手法のプロトタイプは Python 3.12.13 で実装した．実験は MacBook Pro（Intel Core i5 2.0 GHz CPU，32GB RAM，4TB SSD）上で実行した．提案手法の処理 1 の要約および正規化には，OpenAI 社の gpt-5-mini を用いた．Program Dependency Graph の構築には静的解析ツールの joern [16] を用いた．

A.3.2　処理 2 で使用する危険操作 API 辞書の構築

危険操作 API の一覧の取得は，flawfinder [12] の組込みルールを採用した．flawfinder は C/C++ソースコードをスキャンして潜在的なセキュリティ上の欠陥を特定し，その深刻度に応じてランク付けするツールである．プロトタイプ実装では，flawfinder v2.0.20 から取得できた組込みルールの 222 個の関数を，危険操作 API の辞書に採用した．また，SL になりうる標準的な記憶領域に関わる操作の関数（memcpy 等）が網羅されていなかったので，著者らで新たに 27 個の関数を辞書に追加した．

A.4　評価結果の詳細

A.4.1　RQ1 の処理 2 の探索の到達状況

以下の表 A.4.1 は，データセットを用いて提案手法で SL 探索が実行できた FL の割合を示す．

A.5　1 つのデータフローから複数の SL に到達する脆弱性

CVE-2021-26259 と CVE-2021-23206 で，同一のデータ入力処理から複数の SL に到達する脆弱性の構造を表す．

© 2026 Information Processing Society of Japan

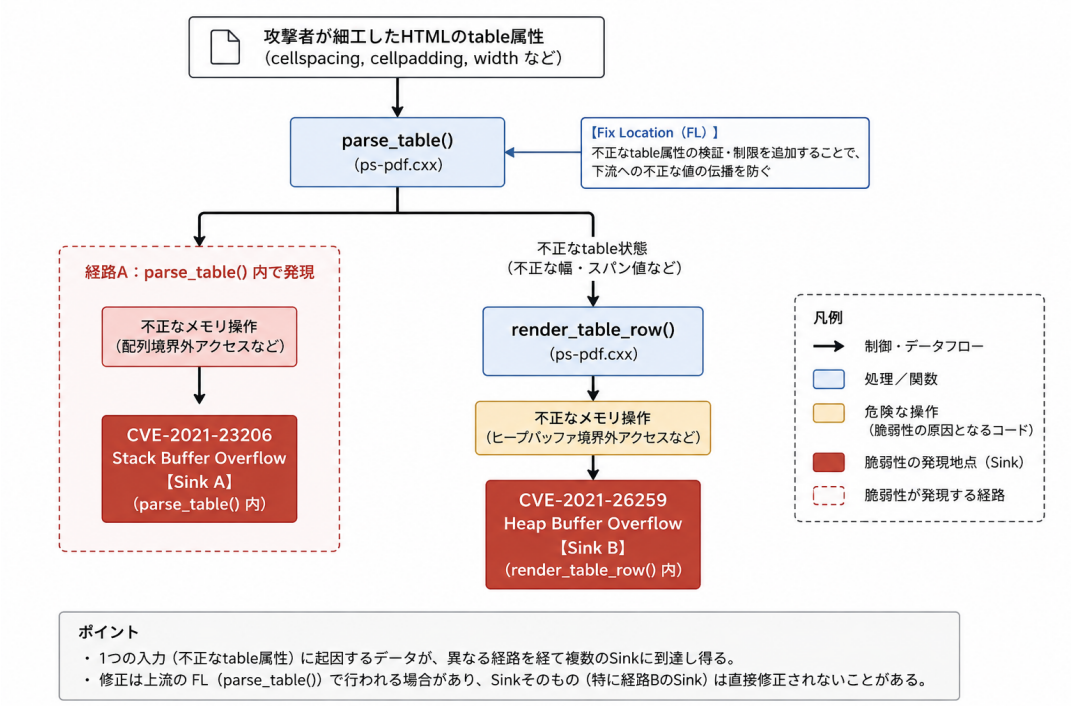


図 A·1　CVE-2021-26259 と CVE-2021-23206 の構造

© 2026 Information Processing Society of Japan